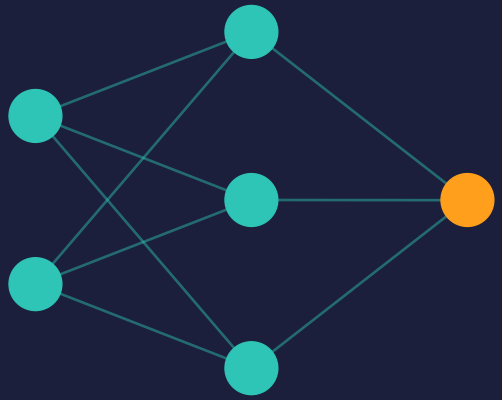
دوره یادگیری عمیق و شبکه‌های عصبی

جلسه ۶

منظم‌سازی و نرمال‌سازی



Early Stopping، BatchNorm، Dropout، Weight Decay و Data Augmentation

کیوان محمودی

در این جلسه

۱ Overfitting در شبکه‌های عصبی

۲ Weight Decay (منظم‌سازی L2)

۳ Dropout

۴ Batch Normalization

۵ LayerNorm

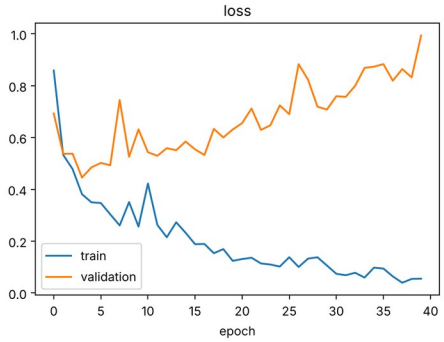
۶ Early Stopping

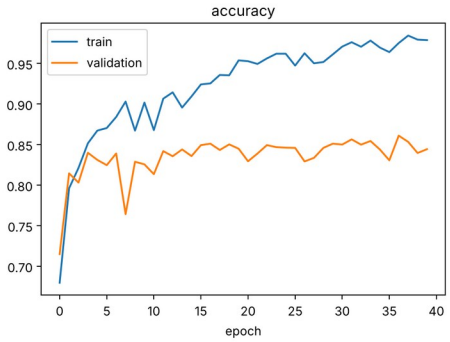
۷ Data Augmentation

۸ مقایسه همه روش‌ها

یک آزمایش کنترل‌شده برای Overfitting

- فقط ۵,۰۰۰ تصویر آموزشی Fashion-MNIST
- MLP با ۲/۹ میلیون پارامتر و ۴۰ Epoch
- دقت آموزش ۹۸٪ و اعتبارسنجی ۸۴٪
- Loss اعتبارسنجی از Epoch چهارم مدام بالا می‌رود.





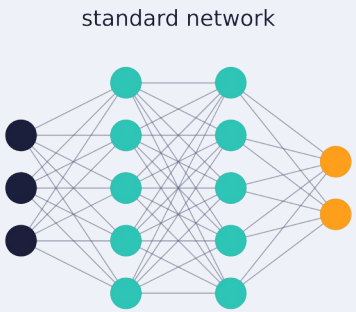
Weight Decay: جریمه وزن‌های بزرگ

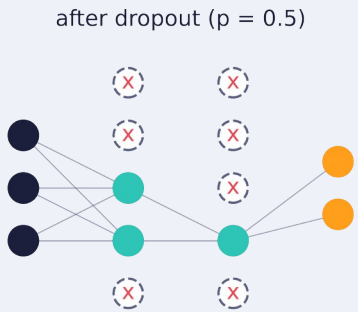
$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \frac{\lambda}{2}\sum_{i} w_i^2$$

$$w \leftarrow (1 - \eta\lambda)\, w - \eta\, \nabla \mathcal{L}_{\text{data}}$$

- در هر گام وزن‌ها کمی به سمت صفر کوچک می‌شوند؛ همان Ridge Regression
- در PyTorch: آرگومان weight_decay در Optimizer
- برای Adam از AdamW استفاده کنید (جلسه ۵).

Dropout: خاموش کردن تصادفی نورون‌ها





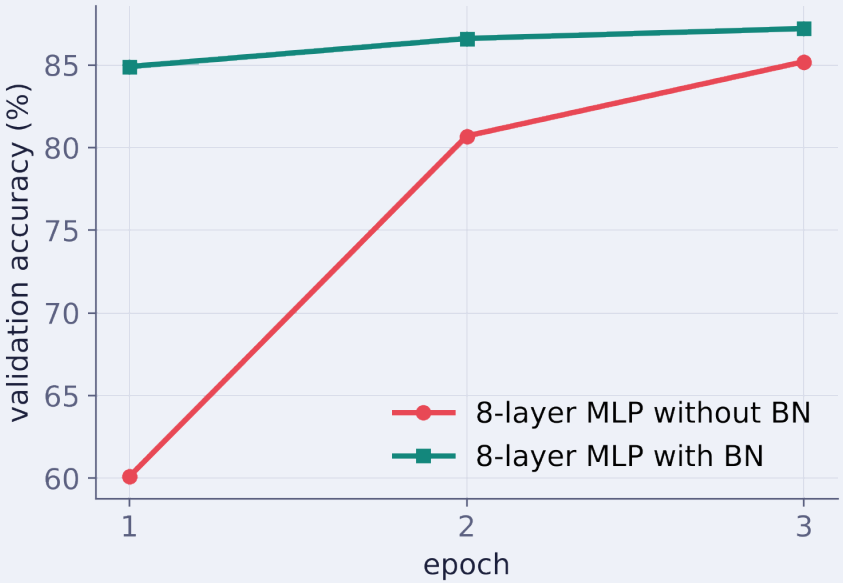
- در هر گام آموزش، هر نورون با احتمال p صفر می‌شود.
- نورون‌ها نمی‌توانند به هم وابسته شوند (Co-adaptation).
- مدل نهایی مثل میانگین تعداد زیادی زیرشبکه عمل می‌کند.
- Inverted Dropout: تقسیم بر (1 - p) در آموزش
- در model.eval خاموش است.

Batch Normalization

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma\,\hat{x}_i + \beta$$

- نرمال‌سازی ورودی هر لایه با میانگین و واریانس همان Batch
- γ و β قابل آموزش‌اند تا شبکه در صورت نیاز توزیع دلخواه را بسازد.
- در آزمون از میانگین متحرک آمار آموزش (Running Stats) استفاده می‌شود.
- پیاده‌سازی از صفر ما دقیقاً با nn.BatchNorm1d برابر شد.

اثر اصلی BatchNorm: سرعت و پایداری



- MLP هشت‌لایه با SGD روی ۵۰,۰۰۰ تصویر
- بعد از یک Epoch: ۶۰٪ بدون BN در برابر ۸۵٪ با BN
- امکان نرخ یادگیری بزرگ‌تر و حساسیت کمتر به مقداردهی اولیه
- در آزمایش ۵,۰۰۰ تصویری، BN جلوی Overfitting را نگرفت.

BatchNorm یا LayerNorm؟

BatchNorm

- نرمال‌سازی روی نمونه‌های Batch، برای هر ویژگی
- رفتار Train و Eval متفاوت است.
- با Batchهای کوچک نویزی می‌شود.
- کاربرد اصلی: CNNها

LayerNorm

- نرمال‌سازی روی ویژگی‌های هر نمونه
- رفتار Train و Eval یکسان است.
- مستقل از اندازه Batch و طول دنباله
- کاربرد اصلی: Transformerها و RNNها

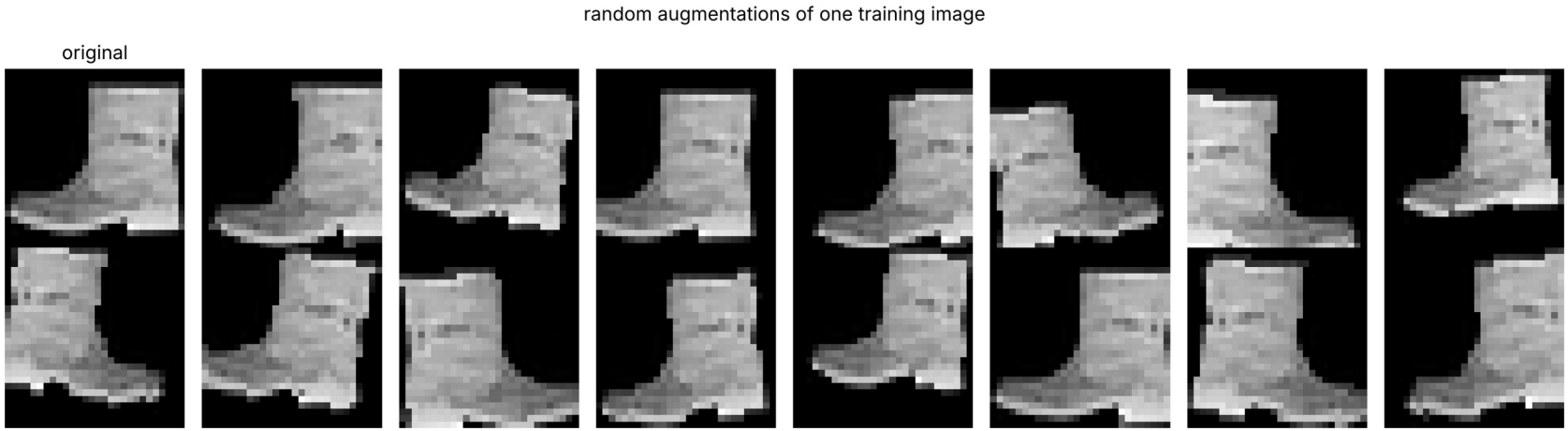
Early Stopping

```python
class EarlyStopping:
    def __init__(self, patience=5):
        self.patience, self.counter = patience, 0
        self.best_loss, self.best_state = float('inf'), None

    def step(self, val_loss, model):
        if val_loss < self.best_loss:
            self.best_loss, self.counter = val_loss, 0
            self.best_state = copy.deepcopy(model.state_dict())
        else:
            self.counter += 1
        return self.counter >= self.patience   # stop?
```

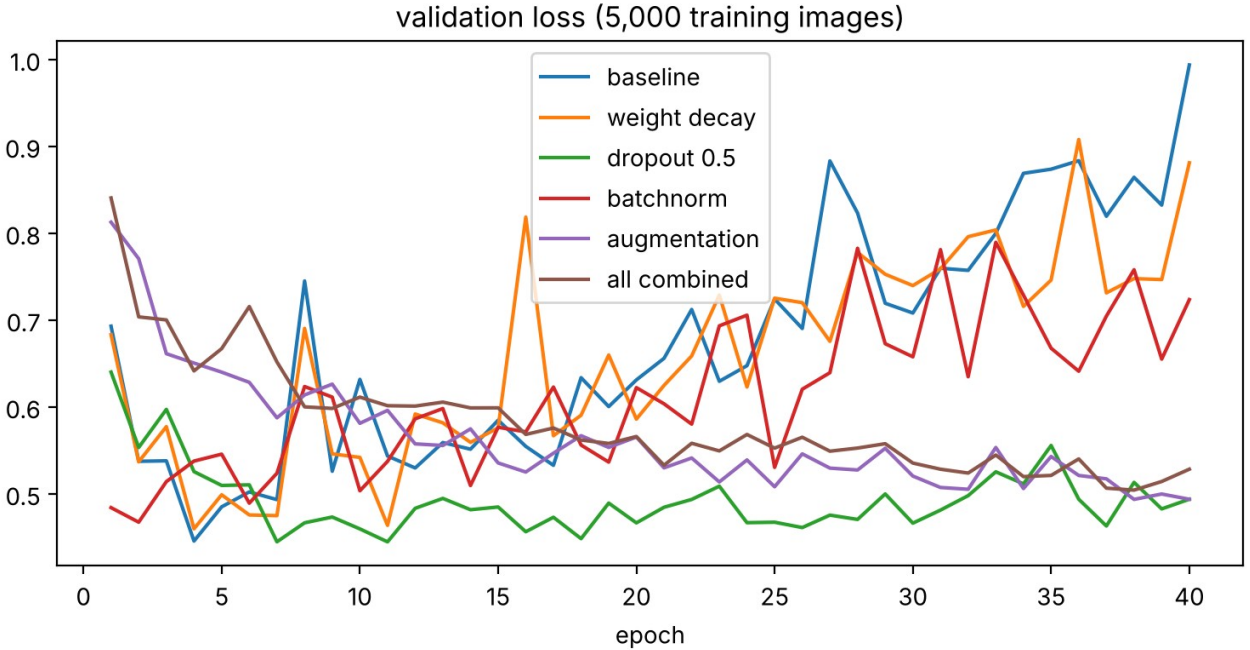
- توقف وقتی Loss اعتبارسنجی برای چند Epoch بهتر نشد
- بازگرداندن بهترین وزن‌ها با load_state_dict
- در آزمایش ما: توقف در Epoch نهم با دقت ۸۴٪

Data Augmentation: داده بیشتر، رایگان



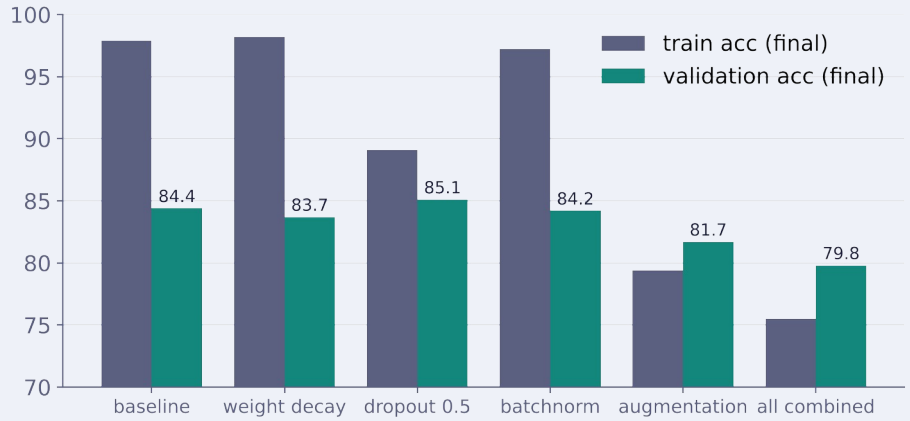
قرینه افقی، جابه‌جایی، چرخش و تغییر مقیاس تصادفی، در هر Epoch و فقط روی داده آموزشی. تبدیل‌ها را با دانش مسئله انتخاب کنید: عدد ۶ چرخیده، ۹ است!

مقایسه: Loss اعتبارسنجی در Epoch ۴۰



Baseline، Weight Decay و BatchNorm منفجر می‌شوند؛ Dropout، Augmentation و ترکیب همه پایدار می‌مانند

نتیجه نهایی و درس‌های آن



- Dropout بهترین تعادل را داد: فاصله دقت فقط ۴٪
- Augmentation و ترکیب همه، Overfitting را حذف کردند اما ۴۰ Epoch برایشان کافی نبود.
- منظم‌سازی قوی‌تر آموزش طولانی‌تر لازم دارد.
- هیچ روشی همیشه بهترین نیست؛ با داده اعتبارسنجی انتخاب کنید.

جمع‌بندی

- Overfitting: فاصله زیاد بین عملکرد آموزش و اعتبارسنجی
- Weight Decay: جریمه L2 روی وزن‌ها؛ برای Adam از AdamW
- Dropout: خاموش کردن تصادفی نورون‌ها در آموزش
- BatchNorm: سرعت و پایداری آموزش؛ LayerNorm برای Transformerها
- Early Stopping و Data Augmentation: ساده و بسیار مؤثر
- model.train و model.eval به‌خاطر Dropout و BatchNorm حیاتی‌اند.

تمرین

۱ Dropout را با p برابر ۰/۱، ۰/۳، ۰/۵ و ۰/۷ تکرار کنید و بهترین دقت را رسم کنید.

۲ Weight Decay را با λ از ۰/۰۰۱ تا ۱ امتحان کنید؛ Underfitting را کجا می‌بینید؟

۳ مدل BatchNorm را بدون model.eval با Batchهای اندازه ۲ ارزیابی کنید؛ چرا دقت افت می‌کند؟

۴ مدل‌های Augmentation و ترکیبی را ۱۵۰ Epoch آموزش دهید؛ آیا از Dropout بهتر می‌شوند؟